VISUAL SPEAKER NOTES • CONTENT-DRIVEN EDITION

Software Engineering Interviews Are Changing

Software Engineering Interviews Are Changing

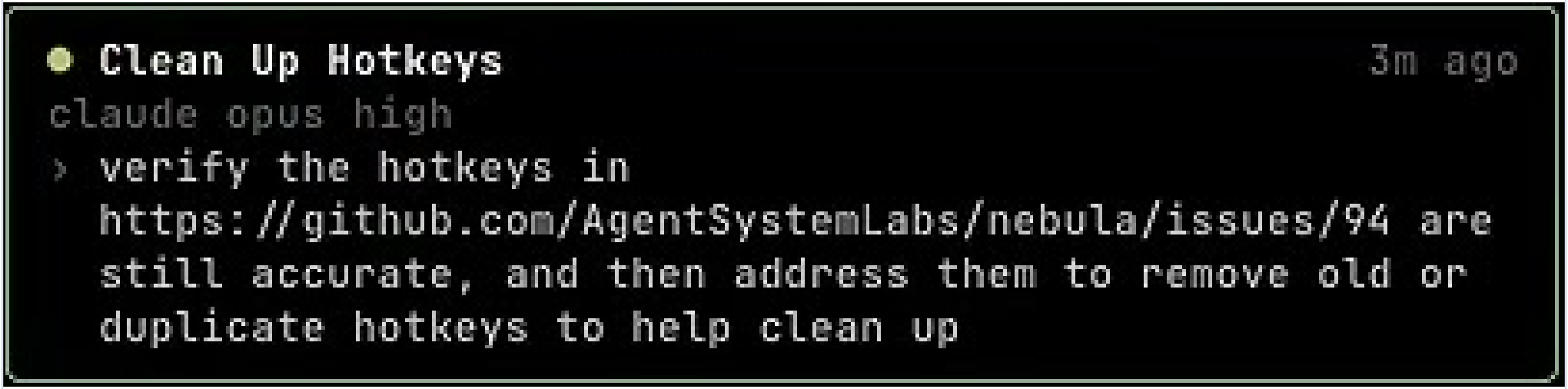
Web Dev Cody • 2026-09-24 • 21:44

- 11 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Speaker claims and caveats belong in the notes

https://www.youtube.com/watch?v=Ge42_DE0rpI

The importance of AI literacy

Agentic-development fluency is becoming interview evidence



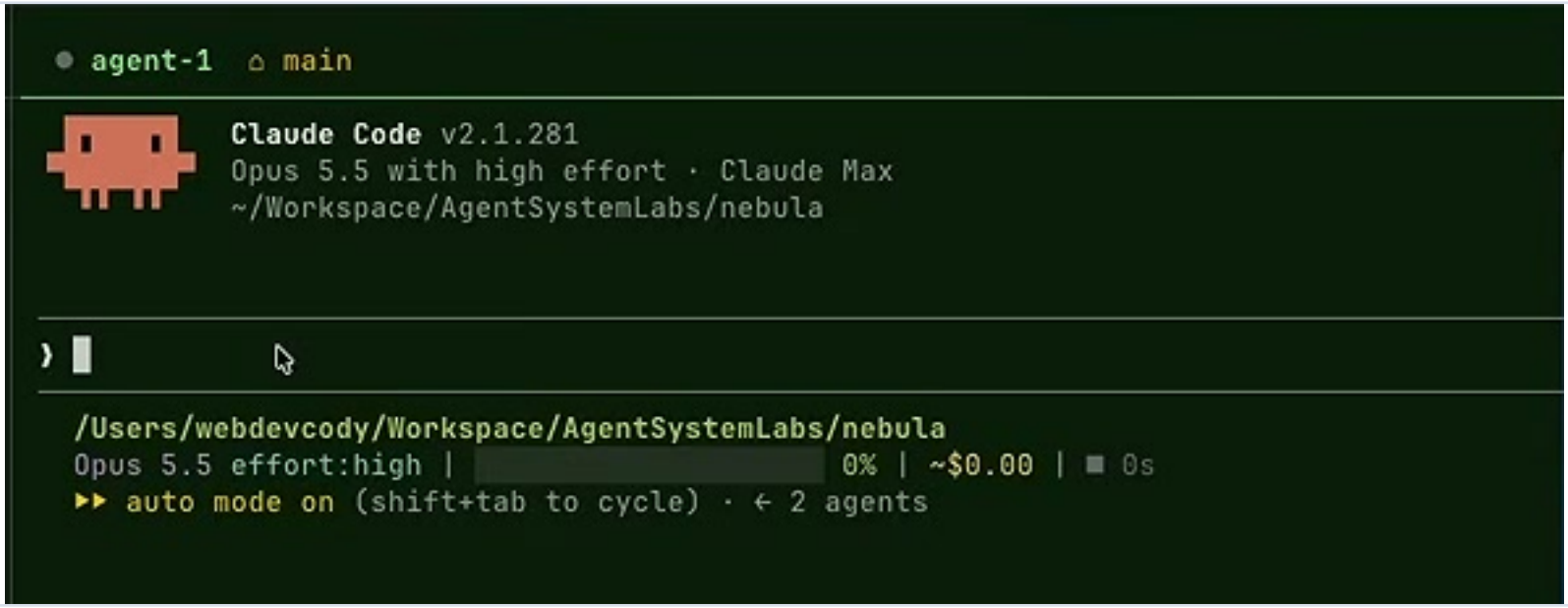
SPEAKER NOTES

WHY IT MATTERS

Interview readiness now includes explaining how AI changes your engineering process, judgment, and output.

Selecting development harnesses

Know more than one interface for agentic coding



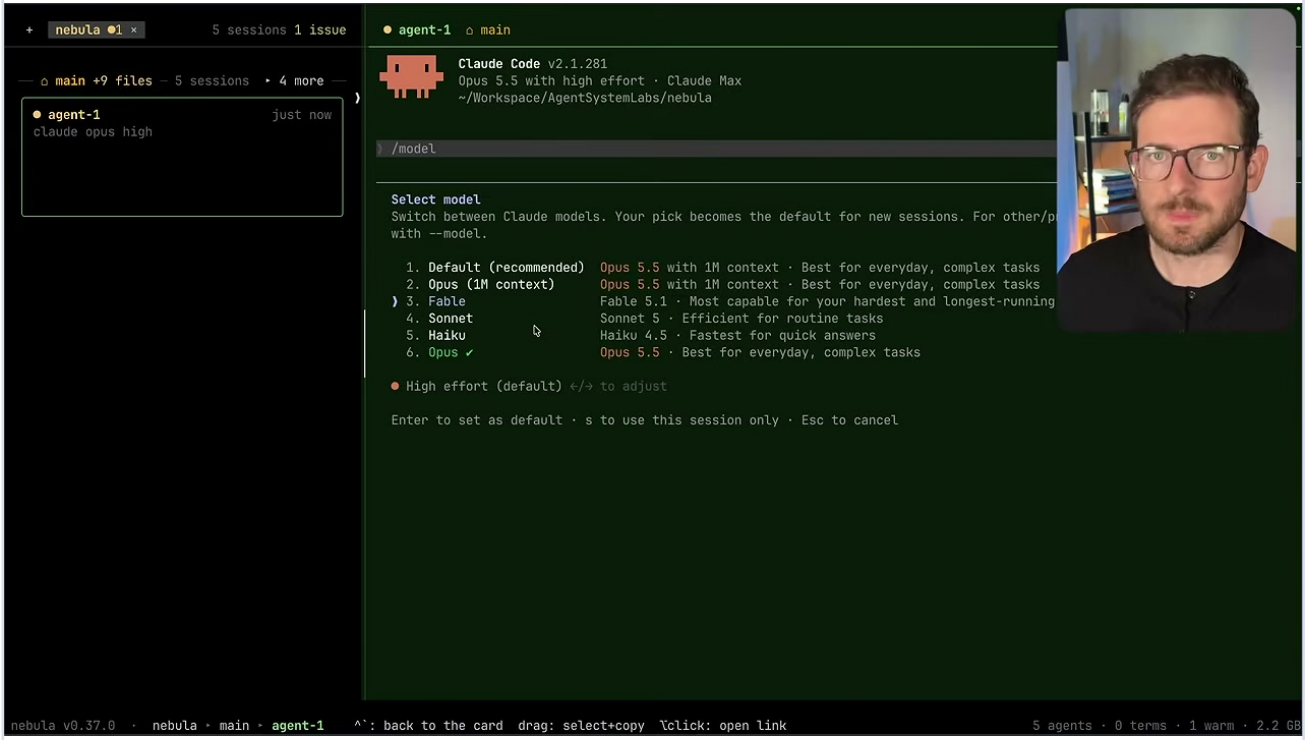
SPEAKER NOTES

WHY IT MATTERS

The harness shapes how effectively you can plan, delegate, inspect, and safely execute work.

Model selection and strategy

Match model capability and cost to the role



SPEAKER NOTES

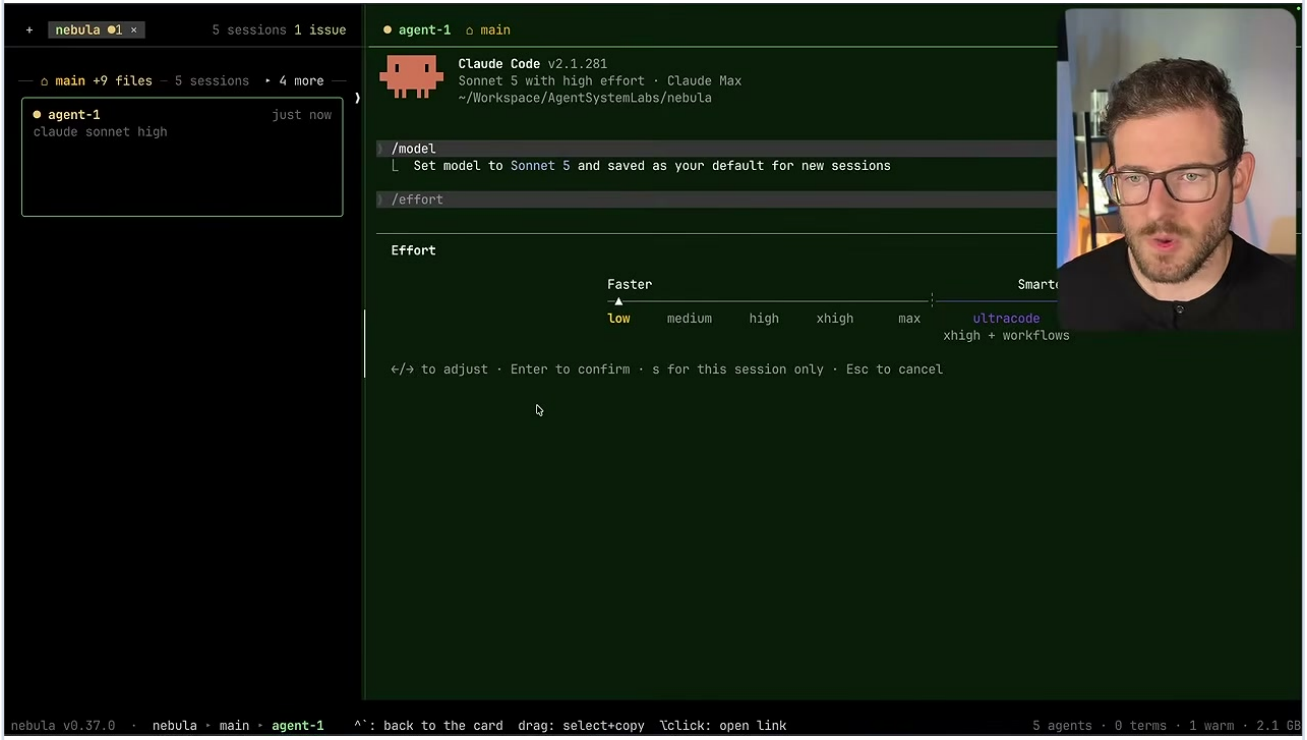
- Candidates should explain when they would choose a long-running frontier model, a strong all-round model, or a cheaper implementation model rather than treating models as interchangeable.
- The speaker uses stronger models for difficult, multi-hour work and orchestration, then delegates bounded implementation tasks to less expensive subagents.
- The tradeoff is explicit: a cheaper model may reduce quality slightly but save substantial tokens when the task is well scoped and reviewed.

WHY IT MATTERS

Model routing is an engineering decision about reliability, latency, context, and budget—not brand preference.

Managing effort levels

Spend reasoning where risk justifies it



SPEAKER NOTES

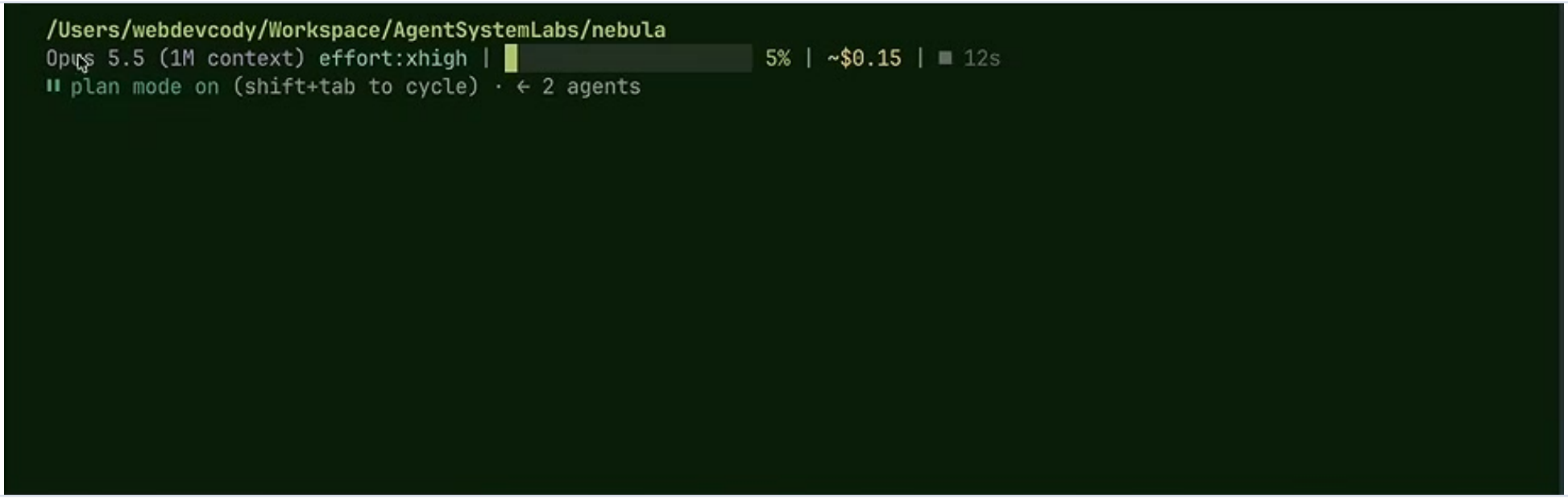
- Low effort suits quick, localized questions that are easy to verify; it is faster and cheaper but more likely to hallucinate.
- Use higher effort for risky refactors and complex integrations, while recognizing the token cost and diminishing returns of maximum settings.

WHY IT MATTERS

Effort level should follow uncertainty and blast radius, not a habit of always maximizing or minimizing reasoning.

Using plan and auto modes

Choose deliberate planning for ambiguity and risk



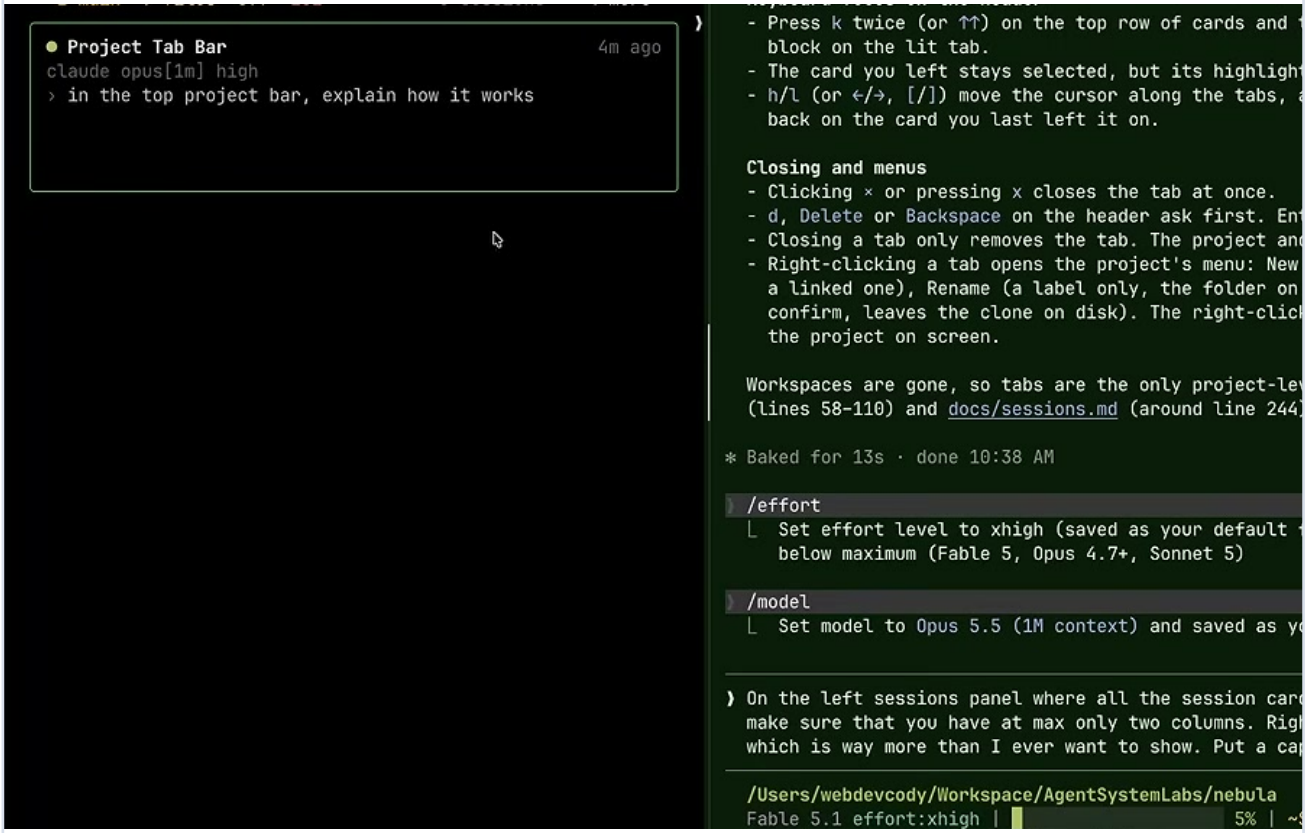
SPEAKER NOTES

WHY IT MATTERS

Planning is a risk-control tool, while automatic permission mediation is an execution-safety tool; they solve different problems.

Effective prompting techniques

Describe where, what, current behavior, and desired behavior



SPEAKER NOTES

- A strong implementation prompt identifies the exact interface area, explains how it behaves now, and states the concrete constraint or result required.
- The example caps a session-card grid at two columns by naming the left sessions panel, the current three-or-four-column behavior, and the desired maximum.
- Voice dictation can communicate richer intent faster than typing, but clarity still depends on precise references and acceptance criteria.

WHY IT MATTERS

Concrete spatial and behavioral context gives the agent less room to guess and makes the result easier to verify.

Creating reusable skills

Encode repeatable team workflows

```
own headings; when you rename or add a heading, add its anchor and recompute the link. This checks a body file:

python3 - <scratchpad>/pr-body.md <<'PY'
import re, sys
body = open(sys.argv[1]).read()
anchors = set(re.findall(r'<a id="([^"]+)"></a>', body))
links = set(re.findall(r'\]\(#([^)]+)\)', body))
unprefixed = sorted(l for l in links if not l.startswith('user-content-'))
targets = {l.removeprefix('user-content-') for l in links}
bare = [h for h in re.findall(r'^## +(.+?)[ \t]*$', body, re.M) if '<a id=' not in h]
print('links missing the user-content- prefix:', unprefixed or 'none')
print('dead TOC links:', sorted(targets - anchors) or 'none')
print('anchors carrying the prefix themselves:', sorted(a for a in anchors if a.startswith('user-content-')) or 'none')
print('## headings without an anchor:', bare or 'none')
PY
```

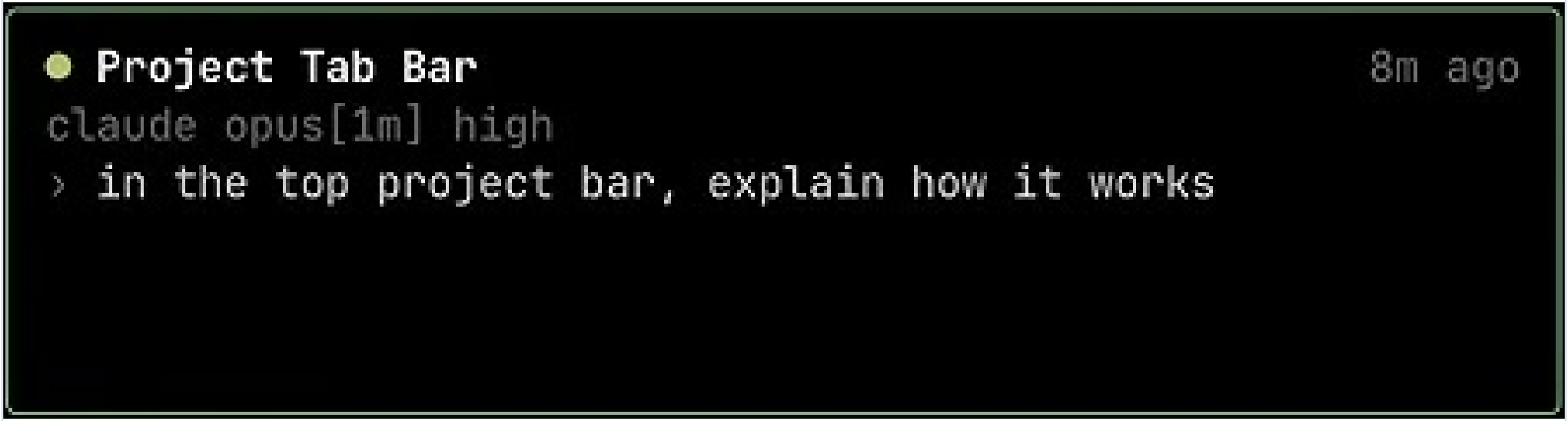
SPEAKER NOTES

WHY IT MATTERS

Skills turn personal prompting knowledge into shared, reviewable team infrastructure.

Leveraging AGENTS.md files

Use always-on context sparingly



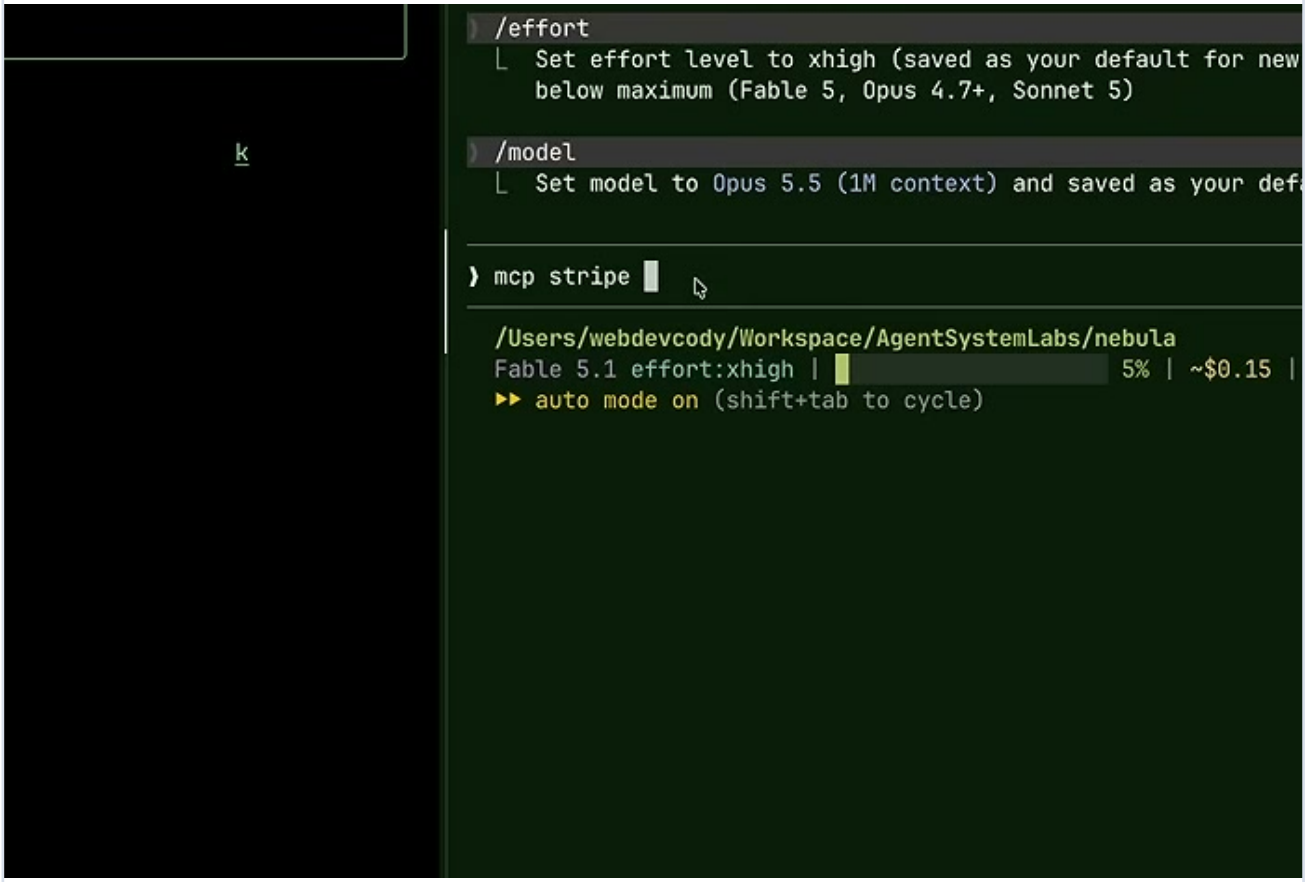
SPEAKER NOTES

WHY IT MATTERS

Keep universal context small; route conditional procedures through on-demand skills.

Utilizing MCP servers

Connect coding agents to external systems



SPEAKER NOTES

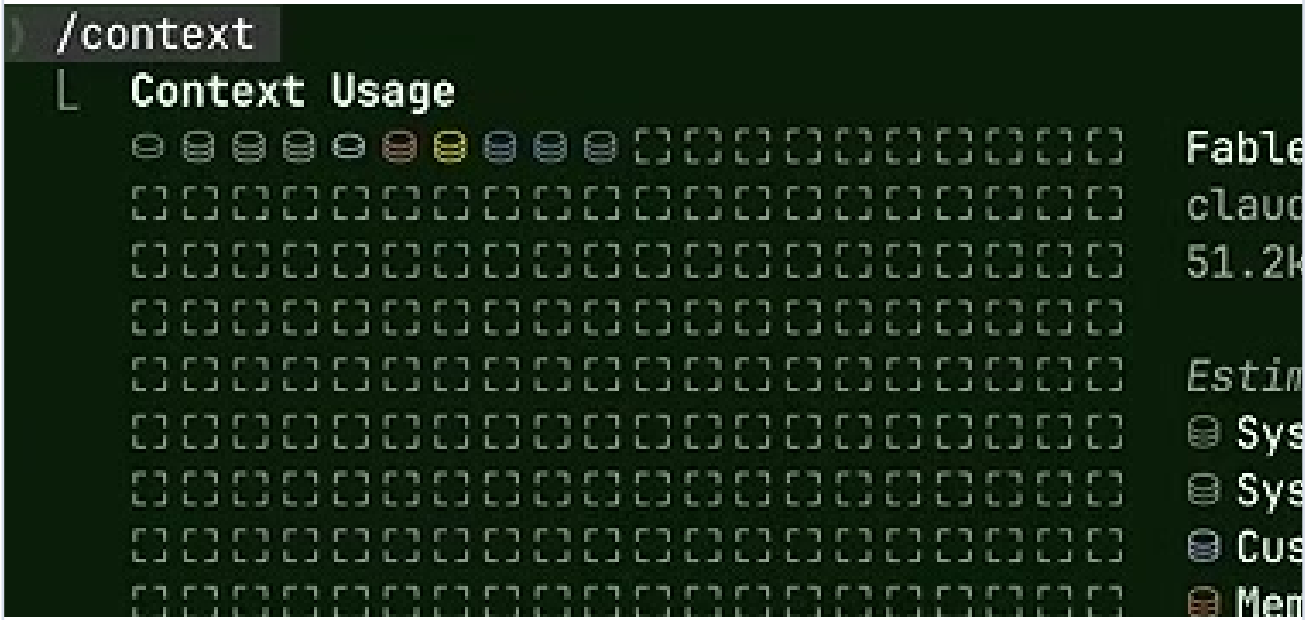
- MCP servers let the agent authenticate to external services and use their capabilities directly from the coding workflow.
- Examples include querying recent Sentry errors, avoiding duplicate issue creation, or configuring Stripe projects and products without repetitive dashboard work.
- The interview signal is practical experimentation: candidates should explain what they connected, what the agent could do, and what safety or authorization boundaries remained.

WHY IT MATTERS

MCP turns the model from a code generator into an operator across the systems involved in software delivery.

Context and hygiene

Inspect what consumes the context window



SPEAKER NOTES

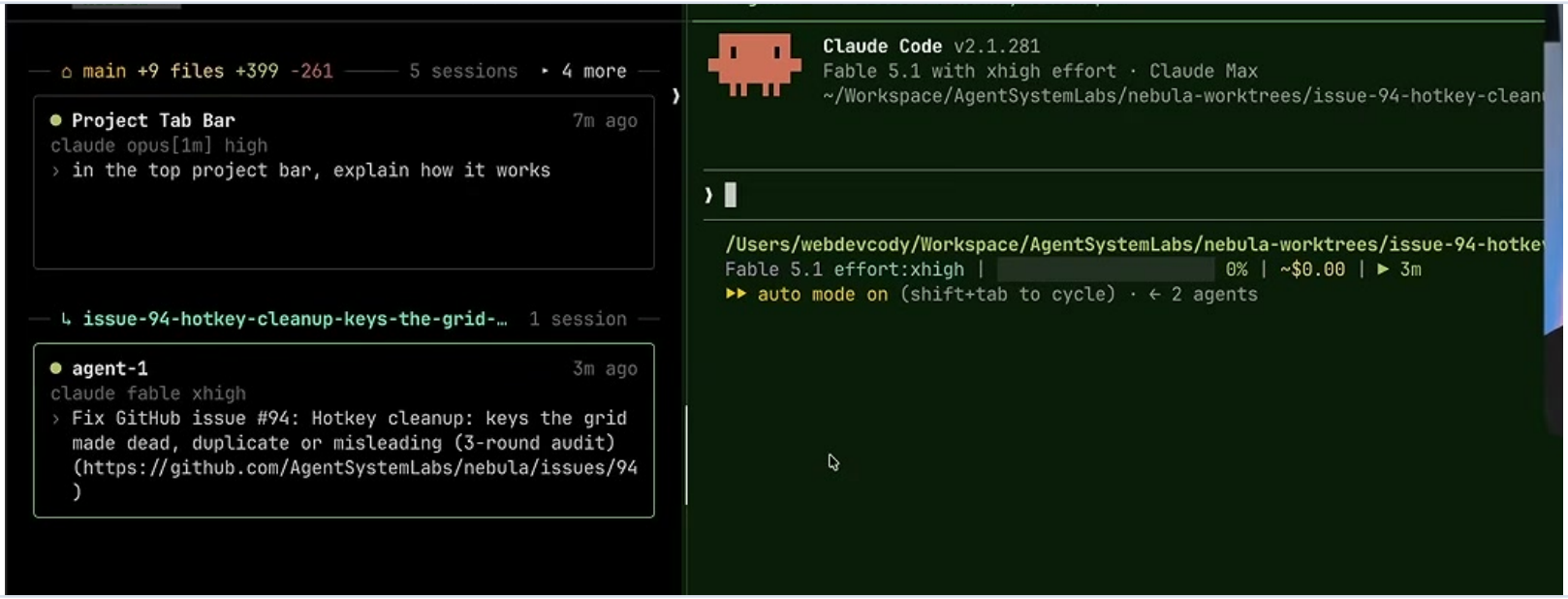
- Context includes the system prompt, tools, agents, memory, skills, MCP schemas, history, and working material.
- Inspect allocation regularly, remove stale integrations, and enable specialized tools only for sessions that need them.

WHY IT MATTERS

Context hygiene improves speed, cost, and model focus by preventing irrelevant instructions and schemas from crowding out the task.

Advanced agentic workflows

Parallelize issues with worktrees, sandboxes, and review guardrails



SPEAKER NOTES

WHY IT MATTERS

The productivity leap comes from safe parallel orchestration—not simply letting one agent run unattended for longer.